

VB.NET

Valider les entrées utilisateur dans une application Console

Mini-cours progressif - nombreux exemples - exercices - procédure générique
finale

Objectif - Savoir empêcher une application Console de poursuivre avec une donnée invalide, distinguer validation technique et validation métier, puis factoriser la saisie dans un helper réutilisable.

Support de cours pratique - VB.NET / Visual Studio 2022

Sommaire

1. 1. Structure du projet à créer
2. 2. Comprendre Console.ReadLine()
3. 3. Parse() : pourquoi il est fragile avec une saisie utilisateur
4. 4. TryParse() : la base de la validation
5. 5. Répéter la saisie avec Do...Loop
6. 6. Valider Integer, Double, Decimal, Date et Boolean
7. 7. Valider le type ET la valeur
8. 8. Valider les chaînes, longueurs, Oui/Non et menus
9. 9. Exercices progressifs avec corrections
10. 10. Factoriser avec ByRef
11. 11. Génériques Of T et inférence de type
12. 12. InputHelper.vb : procédure générique finale
13. 13. Exercice final d'intégration
14. 14. Mémo / erreurs fréquentes

Organisation pédagogique - Les sections 2 à 8 peuvent être faites en live coding. La procédure générique finale doit arriver en dernier, une fois TryParse, les boucles, ByRef et les génériques compris.

1. Structure du projet à créer

Crée un projet Visual Basic de type Application Console. Pour éviter de tout mettre dans Program.vb, on sépare le point d'entrée, les exercices et le code réutilisable.

1.1 Arborescence recommandée

```
ValidationConsole/  
├── Program.vb  
├── InputHelper.vb  
└── Exercices.vb
```

1.2 Rôle de chaque fichier

- Program.vb : contient le seul Main du projet. Il sert de point de départ et appelle les exercices ou le programme final.
- InputHelper.vb : contient les procédures de saisie et de validation réutilisables. On y placera notamment LireValeur(Of T).
- Exercices.vb : contient un Module Exercices avec plusieurs Sub. Chaque exercice est isolé dans une procédure sans créer plusieurs Main.

Important - Ne crée pas un Main dans chaque fichier d'exercice. Dans un même projet Console, conserve un seul point d'entrée et appelle les autres Sub depuis Program.Main.

1.3 Program.vb au démarrage

```
Module Program  
  
    Sub Main(args As String())  
        Console.WriteLine("Cours : validation des entrées")  
    End Sub  
  
End Module
```

1.4 Exercices.vb

```
Module Exercices  
  
    Public Sub ExerciceAge()  
        ' Le code de l'exercice sera placé ici.  
    End Sub  
  
    Public Sub ExerciceNote()  
        ' Le code de l'exercice sera placé ici.  
    End Sub  
  
End Module
```

Pour lancer un exercice depuis Program.vb :

```
Module Program  
  
    Sub Main(args As String())  
        Exercices.ExerciceAge()  
    End Sub  
  
End Module
```

2. Comprendre Console.ReadLine()

Dans une application Console, la saisie utilisateur est généralement lue avec Console.ReadLine(). Le point essentiel est que ReadLine() renvoie une String, même si l'utilisateur tape visuellement un nombre.

```
Console.Write("Votre nom : ")
Dim nom As String = Console.ReadLine()
```

Si l'utilisateur saisit :

```
42
```

VB reçoit d'abord la chaîne de caractères "42". Si le programme a besoin d'un Integer, d'un Double, d'une Date ou d'un Boolean, il faut donc convertir cette chaîne.

Réflexe à retenir - Toute saisie Console commence comme du texte. Avant de l'utiliser comme nombre, date, booléen, etc., le programme doit vérifier qu'elle peut être convertie correctement.

3. Parse() : simple, mais fragile

Une première solution consiste à utiliser Parse().

```
Console.Write("Votre âge : ")
Dim age As Integer = Integer.Parse(Console.ReadLine())
Console.WriteLine("Vous avez " & age & " ans.")
```

La saisie 25 fonctionne. En revanche, une saisie comme "bonjour" ne peut pas être convertie en Integer et provoque une exception de format.

```
Votre âge : bonjour
System.FormatException: The input string was not in a correct format.
```

Conclusion - Parse() est adapté lorsque la donnée est déjà réputée valide. Pour une entrée utilisateur, TryParse() est généralement plus sûr parce qu'il permet de tester la conversion sans faire planter le programme.

4. TryParse() : la base de la validation

TryParse() tente une conversion et renvoie un Boolean : True si la conversion réussit, False sinon. La valeur convertie est placée dans la variable passée en deuxième argument.

```
Dim nombre As Integer
If Integer.TryParse("42", nombre) Then
    Console.WriteLine("Conversion réussie")
    Console.WriteLine(nombre)
Else
    Console.WriteLine("Conversion impossible")
End If
```

4.1 Premier exemple complet

```
Module Program

    Sub Main()

        Dim age As Integer
        Dim saisie As String

        Console.Write("Votre âge : ")
        saisie = Console.ReadLine()

        If Integer.TryParse(saisie, age) Then
            Console.WriteLine("Votre âge est : " & age)
        Else
            Console.WriteLine("Erreur : veuillez saisir un entier.")
        End If

    End Sub

End Module
```

Avec 25, la conversion réussit. Avec toto, le programme affiche le message d'erreur. Il ne plante plus.

```
Votre âge : toto
Erreur : veuillez saisir un entier.
```

5. Redemander tant que la saisie est incorrecte

Le programme ne doit pas seulement détecter l'erreur : dans beaucoup de cas, il doit redemander une valeur jusqu'à ce que la saisie soit valide. Une boucle Do...Loop est très adaptée.

```
Dim age As Integer

Do
    Console.Write("Votre âge : ")

    If Integer.TryParse(Console.ReadLine(), age) Then
        Exit Do
    End If

    Console.WriteLine("Erreur : veuillez saisir un entier.")
Loop
```

```
Votre âge : toto
Erreur : veuillez saisir un entier.
Votre âge : bonjour
Erreur : veuillez saisir un entier.
Votre âge : 25
```

5.1 Variante avec Loop Until

```
Dim age As Integer

Do
    Console.Write("Votre âge : ")
Loop Until Integer.TryParse(Console.ReadLine(), age)

Console.WriteLine("Âge valide : " & age)
```

Cette version est compacte, mais elle donne peu d'informations à l'utilisateur lorsqu'il se trompe. Pour un cours et pour une application conviviale, la version explicite est souvent préférable.

6. Valider les types les plus courants

6.1 Integer

```
Dim age As Integer

Do
    Console.Write("Âge : ")

    If Integer.TryParse(Console.ReadLine(), age) Then
        Exit Do
    End If

    Console.WriteLine("Vous devez saisir un entier.")
Loop
```

6.2 Double

```
Dim prix As Double

Do
    Console.Write("Prix : ")

    If Double.TryParse(Console.ReadLine(), prix) Then
        Exit Do
    End If
```

```
Console.WriteLine("Prix incorrect.")
Loop
```

6.3 Decimal

Pour des montants financiers, Decimal est généralement plus approprié que Double.

```
Dim salaire As Decimal
Do
    Console.Write("Salaire : ")
    If Decimal.TryParse(Console.ReadLine(), salaire) Then
        Exit Do
    End If
    Console.WriteLine("Veuillez saisir un montant valide.")
Loop
```

6.4 Date

```
Dim dateNaissance As Date
Do
    Console.Write("Date de naissance : ")
    If Date.TryParse(Console.ReadLine(), dateNaissance) Then
        Exit Do
    End If
    Console.WriteLine("Date incorrecte.")
Loop
Console.WriteLine("Date enregistrée : " & dateNaissance)
```

6.5 Boolean

```
Dim actif As Boolean
Do
    Console.Write("Actif (True/False) : ")
    If Boolean.TryParse(Console.ReadLine(), actif) Then
        Exit Do
    End If
    Console.WriteLine("Saisissez True ou False.")
Loop
```

Culture - Les séparateurs décimaux et certains formats de date dépendent de la culture du système. Sur un poste français, l'utilisateur saisira souvent 12,5 plutôt que 12.5.

7. Valider le type ET la valeur

Une valeur peut être du bon type mais rester incorrecte pour l'application. Par exemple, 500 est bien un Integer, mais ce n'est probablement pas un âge acceptable.

```
Dim age As Integer
Do
    Console.Write("Votre âge : ")
```

```

Dim saisie As String = Console.ReadLine()

If Integer.TryParse(saisie, age) Then
    If age >= 0 AndAlso age <= 120 Then
        Exit Do
    End If
End If

Console.WriteLine("Veuillez saisir un âge compris entre 0 et 120.")
Loop

```

7.1 Une seule condition avec AndAlso

```

Dim age As Integer

Do
    Console.Write("Votre âge : ")

    If Integer.TryParse(Console.ReadLine(), age) AndAlso
        age >= 0 AndAlso
        age <= 120 Then

        Exit Do
    End If

    Console.WriteLine("Âge incorrect.")
Loop

```

AndAlso est préférable ici car il fait une évaluation conditionnelle. Si TryParse renvoie False, VB n'a pas besoin d'évaluer les autres conditions.

Deux niveaux de validation - Validation technique : « est-ce bien un Integer ? ». Validation métier : « cet Integer est-il acceptable pour mon application ? ». Il faut savoir distinguer ces deux problèmes.

7.2 Exemple : une note entre 0 et 20

```

Dim note As Double

Do
    Console.Write("Note /20 : ")

    If Double.TryParse(Console.ReadLine(), note) AndAlso
        note >= 0 AndAlso
        note <= 20 Then

        Exit Do
    End If

    Console.WriteLine("La note doit être comprise entre 0 et 20.")
Loop

```

8. Valider des chaînes et des choix

8.1 Chaîne non vide

```

Dim nom As String

Do
    Console.Write("Nom : ")
    nom = Console.ReadLine()

```

```

    If Not String.IsNullOrEmpty(nom) Then
        Exit Do
    End If

    Console.WriteLine("Le nom ne peut pas être vide.")
Loop

```

String.IsNullOrEmpty() permet notamment de refuser une chaîne vide ou une chaîne composée uniquement d'espaces.

8.2 Longueur minimale

```

Dim pseudo As String

Do
    Console.Write("Pseudo : ")
    pseudo = Console.ReadLine()

    If Not String.IsNullOrEmpty(pseudo) AndAlso
        pseudo.Length >= 3 Then
        Exit Do
    End If

    Console.WriteLine("Le pseudo doit contenir au moins 3 caractères.")
Loop

```

8.3 Réponse Oui / Non

```

Dim reponse As String

Do
    Console.Write("Continuer ? (O/N) : ")
    reponse = Console.ReadLine().Trim().ToUpper()
Loop Until reponse = "O" OrElse reponse = "N"

If reponse = "O" Then
    Console.WriteLine("Continuation...")
Else
    Console.WriteLine("Arrêt.")
End If

```

8.4 Choix de menu

```

Dim choix As Integer

Do
    Console.WriteLine("1. Addition")
    Console.WriteLine("2. Soustraction")
    Console.WriteLine("3. Multiplication")
    Console.WriteLine("4. Division")
    Console.Write("Votre choix : ")

    If Integer.TryParse(Console.ReadLine(), choix) AndAlso
        choix >= 1 AndAlso choix <= 4 Then
        Exit Do
    End If

    Console.WriteLine("Choisissez une option entre 1 et 4.")
Loop

```

9. Exercices progressifs

Exercice 1 - Âge

Écrire une procédure qui demande l'âge de l'utilisateur. La saisie doit être un entier entre 0 et 120 et doit être redemandée tant qu'elle est incorrecte.

```
Votre âge : toto
Âge incorrect.
Votre âge : -4
Âge incorrect.
Votre âge : 180
Âge incorrect.
Votre âge : 42
Âge enregistré : 42
```

Correction

```
Public Sub ExerciceAge()
    Dim age As Integer

    Do
        Console.Write("Votre âge : ")

        If Integer.TryParse(Console.ReadLine(), age) AndAlso
            age >= 0 AndAlso age <= 120 Then
            Exit Do
        End If

        Console.WriteLine("Âge incorrect.")
    Loop

    Console.WriteLine("Âge enregistré : " & age)
End Sub
```

Exercice 2 - Une note

Demander une note entre 0 et 20. La note doit accepter les décimales.

```
Public Sub ExerciceNote()
    Dim note As Double

    Do
        Console.Write("Note : ")

        If Double.TryParse(Console.ReadLine(), note) AndAlso
            note >= 0 AndAlso note <= 20 Then
            Exit Do
        End If

        Console.WriteLine("Note incorrecte.")
    Loop

    Console.WriteLine("Note enregistrée : " & note)
End Sub
```

Exercice 3 - Calculatrice

Demander deux nombres et une opération. Les nombres doivent être numériques et le menu doit accepter uniquement les choix 1 à 4.

```
Public Sub ExerciceCalculatrice()
    Dim n1 As Double
    Dim n2 As Double
    Dim operation As Integer
```

```

Do
    Console.Write("Nombre 1 : ")
Loop Until Double.TryParse(Console.ReadLine(), n1)

Do
    Console.Write("Nombre 2 : ")
Loop Until Double.TryParse(Console.ReadLine(), n2)

Do
    Console.WriteLine("1. Addition")
    Console.WriteLine("2. Soustraction")
    Console.WriteLine("3. Multiplication")
    Console.WriteLine("4. Division")
    Console.Write("Choix : ")
Loop Until Integer.TryParse(Console.ReadLine(), operation) AndAlso
    operation >= 1 AndAlso operation <= 4
End Sub

```

Exercice 4 - Création d'un utilisateur

Demander un nom, un âge et un salaire. Le nom ne doit pas être vide, l'âge doit être compris entre 18 et 70, et le salaire doit être supérieur ou égal à 0.

```

Public Sub ExerciceUtilisateur()
    Dim nom As String
    Dim age As Integer
    Dim salaire As Decimal

    Do
        Console.Write("Nom : ")
        nom = Console.ReadLine()
    Loop Until Not String.IsNullOrEmpty(nom)

    Do
        Console.Write("Âge : ")
    Loop Until Integer.TryParse(Console.ReadLine(), age) AndAlso
        age >= 18 AndAlso age <= 70

    Do
        Console.Write("Salaire : ")
    Loop Until Decimal.TryParse(Console.ReadLine(), salaire) AndAlso
        salaire >= 0
End Sub

```

Exercice 5 - Mot de passe

Demander un mot de passe d'au moins 8 caractères.

```

Public Sub ExerciceMotDePasse()
    Dim password As String

    Do
        Console.Write("Mot de passe : ")
        password = Console.ReadLine()
    Loop Until Not String.IsNullOrEmpty(password) AndAlso
        password.Length >= 8
End Sub

```

10. Factoriser : première procédure réutilisable

Après quelques exercices, on constate que le même bloc de validation est répété. On peut donc commencer par une procédure dédiée aux Integer.

```
Public Sub LireInteger(message As String, ByRef valeur As Integer)
    Do
        Console.Write(message)

        If Integer.TryParse(Console.ReadLine(), valeur) Then
            Exit Do
        End If

        Console.WriteLine("Valeur incorrecte.")
    Loop
End Sub
```

Utilisation :

```
Dim age As Integer
LireInteger("Votre âge : ", age)
Console.WriteLine(age)
```

11. Comprendre ByRef, Of T et l'inférence

11.1 Pourquoi ByRef ?

Avec ByRef, la procédure reçoit une référence vers la variable d'origine et peut donc la modifier directement.

```
Sub Modifier(ByRef nombre As Integer)
    nombre = 100
End Sub

Dim x As Integer = 10
Modifier(x)
Console.WriteLine(x) ' Affiche 100
```

11.2 Le problème des procédures spécialisées

Si l'on continue avec une procédure par type, on finit avec LireInteger, LireDouble, LireDecimal, LireDate, LireBoolean, etc. Le code devient répétitif.

11.3 Les génériques : Of T

Une procédure générique utilise un paramètre de type T qui sera déterminé à l'appel.

```
Public Sub LireValeur(Of T)(message As String, ByRef valeur As T)
    ' T représente le type demandé.
End Sub
```

Exemples conceptuels :

```
Dim age As Integer
LireValeur(Of Integer)("Âge : ", age)

Dim prix As Double
LireValeur(Of Double)("Prix : ", prix)
```

11.4 Inférence de type

VB peut souvent déduire T grâce à la variable passée en ByRef. On peut alors écrire :

```
Dim age As Integer
LireValeur("Âge : ", age)
```

VB comprend que T vaut Integer parce que age est un Integer.

12. InputHelper.vb : procédure générique finale

On place maintenant la logique réutilisable dans un fichier InputHelper.vb. La procédure suivante sait convertir de nombreux types simples grâce à TypeConverter, avec des traitements particuliers pour String, Char et Enum.

Nuance importante - Aucune procédure ne peut deviner absolument n'importe quel type personnalisé. Cette version fonctionne pour les types courants et pour les types qui disposent d'un TypeConverter adapté. Les objets métier complexes nécessitent généralement une validation spécifique.

```
Imports System.ComponentModel

Module InputHelper

    Public Sub LireValeur(Of T)(
        message As String,
        ByRef valeur As T
    )

        Do
            Console.Write(message)
            Dim saisie As String = Console.ReadLine()

            Try
                ' 1. String
                If GetType(T) Is GetType(String) Then
                    If String.IsNullOrEmpty(saisie) Then
                        Throw New FormatException()
                    End If

                    valeur = DirectCast(CType(saisie, Object), T)
                    Exit Do
                End If

                ' 2. Char
                If GetType(T) Is GetType(Char) Then
                    If saisie Is Nothing OrElse saisie.Length <> 1 Then
                        Throw New FormatException()
                    End If

                    valeur = DirectCast(CType(saisie(0), Object), T)
                    Exit Do
                End If

                ' 3. Enum
                If GetType(T).IsEnum Then
                    Dim resultat As Object = Nothing

                    If [Enum].TryParse(
                        GetType(T),
                        saisie,
                        True,
                        resultat
                    ) Then
                        valeur = DirectCast(resultat, T)
                        Exit Do
                    End If

                    Throw New FormatException()
                End If
            End Try
        Loop
    End Sub

End Module
```

```

        End If

        ' 4. Autres types convertibles depuis String
        Dim converter As TypeConverter =
            TypeDescriptor.GetConverter(GetType(T))

        If converter IsNot Nothing AndAlso
            converter.CanConvertFrom(GetType(String)) Then

            Dim resultat As Object =
                converter.ConvertFromString(saisie)

            valeur = DirectCast(resultat, T)
            Exit Do
        End If

        Catch ex As Exception
            ' La saisie est invalide : la boucle recommence.
        End Try

        Console.WriteLine(
            $"Saisie incorrecte. Type attendu : {GetType(T).Name}"
        )
    Loop

End Sub

End Module

```

12.1 Types courants pris en charge

- String
- Integer, Long, Short, Byte
- Double, Single, Decimal
- Boolean
- Date
- Char
- Enum
- Autres types exposant un convertisseur compatible depuis String

12.2 Utilisation dans Program.vb

```

Module Program

    Sub Main()
        Dim nom As String
        Dim age As Integer
        Dim taille As Double
        Dim salaire As Decimal
        Dim naissance As Date
        Dim actif As Boolean
        Dim initiale As Char

        LireValeur("Nom : ", nom)
        LireValeur("Âge : ", age)
        LireValeur("Taille : ", taille)
        LireValeur("Salaire : ", salaire)
        LireValeur("Date de naissance : ", naissance)
        LireValeur("Actif (True/False) : ", actif)
        LireValeur("Initiale : ", initiale)

        Console.WriteLine()
        Console.WriteLine("----- Résultat -----")
        Console.WriteLine("Nom : " & nom)
        Console.WriteLine("Âge : " & age)
        Console.WriteLine("Taille : " & taille)
    End Sub
End Module

```

```

        Console.WriteLine("Salaire : " & salaire)
        Console.WriteLine("Naissance : " & naissance)
        Console.WriteLine("Actif : " & actif)
        Console.WriteLine("Initiale : " & initiale)
    End Sub
End Module

```

12.3 Exemple avec Enum

```

Enum Role
    Administrateur
    Formateur
    Etudiant
End Enum

Module Program

    Sub Main()
        Dim roleUtilisateur As Role

        LireValeur("Rôle : ", roleUtilisateur)
        Console.WriteLine(roleUtilisateur)
    End Sub

End Module

```

Grâce au paramètre True dans Enum.TryParse, la casse est ignorée : Formateur, FORMATEUR ou formateur peuvent être interprétés comme le même membre de l'énumération.

12.4 Ce que LireValeur ne doit pas faire

Le helper vérifie surtout si la saisie peut devenir le type demandé. Il ne connaît pas toutes les règles métier de l'application.

```

Dim age As Integer
LireValeur("Âge : ", age)

If age < 0 OrElse age > 120 Then
    Console.WriteLine("Âge impossible.")
End If

```

La séparation à retenir est :

```

SAISIE UTILISATEUR
    ↓
VALIDATION DU TYPE
    ↓
VALIDATION MÉTIER
    ↓
UTILISATION DE LA DONNÉE

```

13. Exercice final d'intégration

Créer une application d'inscription qui demande les informations suivantes en utilisant LireValeur(...) pour chaque saisie :

- Nom : String
- Prénom : String
- Âge : Integer

- Taille : Double
- Salaire : Decimal
- Date de naissance : Date
- Statut actif : Boolean

Ajouter ensuite les règles métier suivantes :

- Nom non vide
- Prénom non vide
- Âge entre 18 et 100
- Taille entre 1 et 2,5 mètres
- Salaire supérieur ou égal à 0
- Date de naissance antérieure à aujourd'hui

Exemple d'affichage final :

```
-----
      FICHE UTILISATEUR
-----
Nom       : Dupont
Prénom    : Alice
Âge       : 34
Taille    : 1,68
Salaire   : 3200
Naissance : 14/05/1992
Actif     : True
```

13.1 Piste de correction : saisies

```
Dim nom As String
Dim prenom As String
Dim age As Integer
Dim taille As Double
Dim salaire As Decimal
Dim naissance As Date
Dim actif As Boolean

LireValeur("Nom : ", nom)
LireValeur("Prénom : ", prenom)
LireValeur("Âge : ", age)
LireValeur("Taille : ", taille)
LireValeur("Salaire : ", salaire)
LireValeur("Date de naissance : ", naissance)
LireValeur("Actif (True/False) : ", actif)
```

14. Mémo et erreurs fréquentes

14.1 À retenir

- Console.ReadLine() renvoie une String.
- Parse() convertit, mais peut lever une exception si la saisie n'est pas valide.
- TryParse() est généralement le meilleur réflexe pour une saisie utilisateur simple.
- Do...Loop permet de redemander une valeur jusqu'à ce qu'elle soit correcte.
- AndAlso permet de combiner proprement une validation de type et une validation de plage.
- String.IsNullOrEmpty() permet de refuser un texte vide ou rempli d'espaces.
- ByRef permet à une procédure de renseigner directement une variable fournie par l'appelant.
- Of T rend la procédure générique et VB peut souvent inférer T automatiquement.
- Validation technique et validation métier sont deux étapes différentes.

14.2 Erreurs fréquentes

- Utiliser `Integer.Parse(Console.ReadLine())` sans gérer une mauvaise saisie.
- Valider uniquement le type mais oublier les bornes métier, par exemple accepter un âge de 500.
- Créer plusieurs `Main` dans le même projet pour les exercices.
- Dupliquer les mêmes boucles partout au lieu de factoriser lorsque la notion est comprise.
- Introduire trop tôt la procédure générique : les élèves doivent d'abord comprendre `TryParse` et les boucles.

Progression conseillée - 1) `ReadLine` → 2) `Parse` → 3) `TryParse` → 4) `Do...Loop` → 5) règles métier → 6) `ByRef` → 7) `Of T` → 8) `InputHelper` générique. Cette progression rend le besoin de factorisation naturel.

Annexe - Fichiers finaux du projet

A. Program.vb

```
Module Program

    Sub Main(args As String())
        Dim nom As String
        Dim age As Integer
        Dim salaire As Decimal

        LireValeur("Nom : ", nom)
        LireValeur("Âge : ", age)
        LireValeur("Salaire : ", salaire)

        Console.WriteLine()
        Console.WriteLine("Nom : " & nom)
        Console.WriteLine("Âge : " & age)
        Console.WriteLine("Salaire : " & salaire)
    End Sub

End Module
```

B. InputHelper.vb

```
Imports System.ComponentModel

Module InputHelper

    Public Sub LireValeur(Of T)(message As String, ByRef valeur As T)
        Do
            Console.Write(message)
            Dim saisie As String = Console.ReadLine()

            Try
                If GetType(T) Is GetType(String) Then
                    If String.IsNullOrEmpty(saisie) Then
                        Throw New FormatException()
                    End If
                    valeur = DirectCast(CType(saisie, Object), T)
                    Exit Do
                End If

                If GetType(T) Is GetType(Char) Then
                    If saisie Is Nothing OrElse saisie.Length <> 1 Then
                        Throw New FormatException()
                    End If
                    valeur = DirectCast(CType(saisie(0), Object), T)
                    Exit Do
                End If

                If GetType(T).IsEnum Then
                    Dim resultat As Object = Nothing
                    If [Enum].TryParse(GetType(T), saisie, True, resultat) Then
                        valeur = DirectCast(resultat, T)
                        Exit Do
                    End If
                    Throw New FormatException()
                End If

                Dim converter As TypeConverter =
                    TypeDescriptor.GetConverter(GetType(T))

                If converter IsNot Nothing AndAlso
                    converter.CanConvertFrom(GetType(String)) Then

                    Dim resultat As Object =
                        converter.ConvertFromString(saisie)

                    valeur = DirectCast(resultat, T)
                End If
            Catch
            End Try
        Loop
    End Sub

End Module
```

```

        Exit Do
    End If

    Catch ex As Exception
        ' Saisie invalide : on recommence.
    End Try

    Console.WriteLine(
        $"Saisie incorrecte. Type attendu : {GetType(T).Name}"
    )
Loop
End Sub
End Module

```

C. Exercices.vb

```

Module Exercices

    Public Sub ExerciceAge()
        Dim age As Integer

        Do
            Console.Write("Votre âge : ")

            If Integer.TryParse(Console.ReadLine(), age) AndAlso
                age >= 0 AndAlso age <= 120 Then
                Exit Do
            End If

            Console.WriteLine("Âge incorrect.")
        Loop

        Console.WriteLine("Âge enregistré : " & age)
    End Sub

    Public Sub ExerciceNote()
        Dim note As Double

        Do
            Console.Write("Note : ")

            If Double.TryParse(Console.ReadLine(), note) AndAlso
                note >= 0 AndAlso note <= 20 Then
                Exit Do
            End If

            Console.WriteLine("Note incorrecte.")
        Loop

        Console.WriteLine("Note enregistrée : " & note)
    End Sub
End Module

```